

CUDA PROGRAMMING FOR NVIDIA H100S

第 1 课 - H100 简介

Prateek Shukla · 简体中文版

规格

- 2022 年 9 月发布 | 基于 TSMC 4N 工艺节点
- 基于 Hopper 架构
- 两种形态：PCIe(300W) 和 SXM(700W)
- 80 GB HBM 3 内存 (3.35 TB/s)
- 132 个 SM
- 528 个张量核心 (每个 SM 4 个)
- 定制 4N 节点上的 800 亿个晶体管

最大的改进是引入了异步特性

TMA

每个 SM 都有一个 TMA 单元。这是用于把张量拷贝操作从 SM 上卸载出去的新单元。

它的诞生是为了加速内存受限 (memory-bound) 操作

以前我们必须做每线程的数学运算、循环遍历区域、发射多条指令，才能完成全局内存 -> 共享内存的传输

有了 TMA，我们得到描述符 (descriptor) 来异步地做这些事。由单个线程发起整个数据搬运操作。

硬件在后台处理步长/偏移/边界以及数据搬运

同步拷贝 vs 异步拷贝

如果发起拷贝的线程（或 warp）在拷贝完成之前无法继续向前推进，那这个拷贝就是同步的。它的心智模型很简单：你知道一旦控制权回到线程，操作就已完成

如果发起拷贝只是启动传输，而发起者可以在拷贝在途期间继续做其他工作，之后在使用数据之前再等待/检查完成，那这个拷贝就是异步的。

这非常棒，因为它实现了重叠（把内存延迟隐藏在计算之下）。对流水线化 kernel（GEMM/attention）更好：在计算 tile i 的同时，取回 tile $i+1$ 。

唯一的问题是簿记复杂度增加

第四代张量核心

每个 SM 4 个张量核心

Warp Group Matrix Multiply Accumulate (wgmma) —— 4 个 warp 共同完成一次 MMA

稀疏张量核心，可以对结构化稀疏数据（在预定义布局中含有零的数据）执行非常快的 MMA 操作

针对 FP8 的特殊指令与支持。可提供 1,979 TFLOPS（比上一代 GPU 有巨大提升）

FP16 为 1024 FLOPS/cycle（稠密），FP8 为 2048 FLOPS/cycle。

其他内容

- Transformer Engine
- 第 4 代 NVLink 互连
- DPX 指令
- 巨大的 50MB L2 缓存

GPU 的整体组织

一整块 GPU 芯片 (die) 被划分为——

- Nvidia Gigathread Engine
- 8 个 GPC
- HBM3 堆栈
- HBM3 内存控制器
- PCI express 5.0 主机接口
- Nvlink 交换机、端口、集线器
- L2 缓存切片

GigaThread Engine

GigaThread Engine 是接收 kernel 启动并把线程块（CTA）分发给 SM 的硬件。

它跟踪哪些 CTA 尚未启动、哪些正在运行、哪些已完成。当某个 SM 有容量再容纳一个 CTA 时，GigaThread Engine（以及相关的前端逻辑）就把下一个 CTA 分配给那个 SM。

它执行诸如占用率上限之类的约束，而且在 Hopper 上它还理解集群（下文详述）。

HBM3

片外设备内存。80GB , 3.35 TB/sec

分为 5 个堆栈（实际是 6 个，但出于良率原因禁用了一个）

5120 位总线宽度，使 TMA 能一次搬运 128 字节数据

通过 10 个独立的 512 位内存控制器连接

SM -> (L1D / 合并器 (coalescer)) -> L2 -> 内存分区/交叉开关 (crossbar) -> 内存控制器 -> HBM 堆栈

GPU 内存控制器块在做关键工作：

DRAM 协议 + 调度 (activate/precharge/read/write 时序，为最大化行命中而重排序)。地址映射 -> 通道 /bank/行/列

L2 缓存

50 MB，划分为 25 MB 的分区L2 缓存是分区的，给定 GPC 中的 SM 到它直连的 L2 分区有一条"更近"的路径。当你的访问大多命中驻留在邻近分区中的缓存行时，你会得到更好的有效延迟/带宽

128 字节缓存行和 32 字节扇区 (sector)。一个内存请求可以触及单个 128B 行中的 1-4 个扇区。你可能以某种让扇区数/请求数爆炸的方式"未合并"。

它吸收并合并来自 SM (经由 L1) 的杂乱小写入，把它们变成干净、大块、高效的到 HBM 的写入。

GPC - 图形处理集群 (Graphics Processing Cluster)

它是一组 18 个 SM。

每个 GPC 都连接到属于自己的一块 L2 缓存。当有请求要把数据取到共享内存时，数据从 HBM -> L2 -> L1 流动，而 GPC 负责处理所有这些

GPC 还让 SM 之间可以使用分布式共享内存 (DSMEM)。GPC 之外没有 DSMEM

GPC 里有 TPC，它基本上容纳 2 个 SM，其唯一的工作就是确保 TPC 内部 SM 之间的通信非常快。这条连接就是为 DSMEM 服务的

NVLink 4.0

18 条 NVLink 4.0 通道 (lane) , 提供 900 GB/s 的 GPU - GPU 带宽

组织成 9 个子链路 (sub-link) , 每个提供 100 GB/s 双向带宽 (每方向 50GB/s)

PAM4 信令 , 基本上是使用 4 个不同的电压电平而不是 2 个。这使 GPU 之间每秒传输的比特数翻倍

你可以直接连接 8 个 GPU , 并用 NVSwitch 交换网络 (fabric) 连接多达 256 个 GPU

与 TMA 直接相连 , 在单个 DGX 节点内进行批量拷贝

SM

GPU 的基础执行单元，执行 CUDA kernel 的线程块。以下是它的组成部分——

- FP32 CUDA 核心、Int/FP64 单元
- 第四代张量核心
- 共享内存/L1 缓存
- L1 指令缓存
- Warp 调度器
- 分发单元
- 寄存器
- L0 指令缓存

象限 (**Quadrant**) /SMSP

每个 SM 被划分为 4 个完全相同的子分区，称为象限 (Quadrant) 或 SMSP

每个象限有 16 个 FP64 单元、32 个 FP32 CUDA 核心、16 个 INT32 单元、1 个张量核心、8 个加载/存储 (load/store) 单元、4 个特殊功能单元 (SFU)

除此之外，每个象限还有自己的 warp 调度器、L0 缓存、分发单元和寄存器堆 (register file)

每个象限每个周期都可以向它的本地单元分发指令，每个象限最多要兼顾 16 个 warp

特殊功能单元 (**SFU**)

每个 SM 有 16 个 SFU (每个 SMSP/象限 4 个)。每个 SFU 每周期发射 1 条指令，即每个 SM 每周期 16 次操作

SFU 处理正弦、余弦、对数、指数、平方根和倒数这类复杂数学函数，这些函数在普通核心上计算代价很高

SFU 用多项式近似和查找表加插值来换取巨大的吞吐量提升，牺牲了一些精度

如果所有线程都调用数学函数，SFU 会停顿 (stall)

加载/存储单元

它们执行每线程的内存指令，如加载、存储和原子操作（atomic）

每个象限包含 8 个专用的加载/存储单元，每个 SM 共 32 个 LSU，它们直接连接到 L1、L2

当你的 warp 执行 ld/st/atom 时，LSU 合并 32 个线程地址，构成缓存行/扇区请求，并查询 L1 数据缓存。命中时，数据很快回到寄存器；未命中时，请求继续前往 L2/DRAM，并可能回填进 L1。

当访问被合并（coalesced）时，LSU 把 warp 的 32 个地址合并进尽可能少的缓存行，因此它向 L1/L2 发出更少的请求，减少重放（replay）/停顿并提升有效带宽。

统一共享内存 + L1 缓存

每 SM 256 KB 大小、33 TB/s 带宽，划分为 32 个 bank，每个 bank 32 位（4 字节）宽，共享内存大小为 228kb 加载和存储为 128B/周期，无冲突。TMA 异步拷贝达到接近峰值带宽并与计算重叠。bank 把连续的 4 字节字交错（interleave）分布到顺序的各个 bank 上。L1 缓存充当合并缓冲：它收集 warp 请求的数据并高效交付缓存行 = 128 字节，扇区 = 32 字节，缓存可以按扇区逐个填充每 SM 可配置的最大共享内存容量为 228 KB 由于 CUDA 保留了 1 KB，每线程块上限为 227 KB

关于共享内存的更多内容

字节地址包含关于 bank、bank 内行 (bankrow) 等信息

静态共享内存 (编译期数组声明) 在架构上被限制为 48 KB, 以保持与旧计算能力的兼容。要超过这个限制, 你需要使用动态共享内存, 方式是

```
extern __shared__
```

对 GEMM 而言, 最优 tile 大小通常为每块 64-128 KB。巨大的传输会伤害延迟隐藏 (latency hiding)

所以如果你把共享内存调大, 留给 L1 缓存的空间就被压缩, 反之亦然。

INT 单元

这些是标准的整数算术逻辑单元，用于内存寻址、循环控制和一般整数数学。

架构：每个 SM 有 64 个 INT32 核心。

并发执行：这些单元可以与浮点数据通路同时执行，让 GPU 在处理数据（FP）的同时计算内存地址（INT）而不停顿。

总数（SXM5）：8,448 个单元。

FP32 CUDA 核心

NVIDIA 营销术语中的"CUDA 核心"一般指单精度 (FP32) 浮点单元。在 Hopper 中, 它们是通用图形与计算着色的主力。

H100 中的每个流式多处理器 (SM) 包含 128 个 FP32 CUDA 核心。

总数 (SXM5) : 16,896 个活跃核心 (启用 132 个 SM)。

它们处理标准的 32 位浮点计算, 对大多数科学模拟至关重要, 也是通用计算任务的"默认"精度。

FP64 单元

H100 具有独立于 FP32 核心的专用 FP64 核心。每个 SM 有 64 个 FP64 核心。

一般用于科学计算等。

寄存器

每个线程都有一组私有的片上寄存器。它们带宽最快、延迟最低，每 SM 256 KB。每 SM 每周期最多 128 次读/写。

寄存器是常见的限制因素。如果你的 kernel 每线程使用 R 个寄存器，那么最大驻留线程数约为 $\text{floor}(65536 / R)$ （之后还要按块粒度和其他限制取整/受限）。每线程 128 个寄存器时，你被限制为每块 512 线程。

32 位是寄存器的基本单位。硬件寄存器堆实际上是 32 位槽位。使用 FP16 或 FP8 时，我们必须使用打包数据类型，才能把 2/4 个元素装进单个寄存器

关于寄存器的更多内容

寄存器堆访问的能效比共享内存访问高 30-50 倍，比 HBM3 访问高 1000 倍以上！

寄存器用量在编译期确定，而不是运行时

当编译器用完寄存器时，它会把数据溢出（spill）到本地内存（local memory），这比寄存器慢得多。CUDA 13.0 增加了对先溢出到共享内存的支持，只有当共享内存不足时才回退到本地内存。

即使很小的变化（比如 63 -> 65 寄存器/线程）也可能减少驻留 warp，因为分配会按内部粒度取整。你会看到更少的活跃 warp 和更低性能

WGMMMA 寄存器布局

张量核心操作需要特定的寄存器布局：

矩阵 A：存放在共享内存或寄存器中

矩阵 B：必须始终在共享内存（SMEM）中，绝不能在寄存器中

矩阵 C：以寄存器形式分布在 4 个 warp（128 线程）上

使用 FP8/FP16 时，WGMMMA 要求数据被打包进寄存器，即单个 32 位寄存器装 2 个 FP16。

L0 指令缓存

L0 指令缓存 (I-Cache) 充当指令流的微型缓冲。它的目的是过滤发往更大 L1 缓存的请求，让调度器能以硬件的速度（每个时钟周期）发射指令，而不会让共享内存带宽饱和。

L0 缓存为速度而设计，而不是容量。它只能容纳非常少量的指令（通常只装得下几个紧凑循环）。激进的循环展开、函数内联可能会超出 L0 缓存的大小。

运行在 SMSP 0 上的代码不能使用 SMSP 1 的 L0 缓存。

什么是 **warp**

warp 是一组 32 个线程，它们：

从同一个线程块一起创建（线程 0-31 = warp 0，32-63 = warp 1）

按周期共享一个 warp 调度器的关注

拥有私有寄存器，但（大体上）共享指令流

为什么是 32 线程？

每条指令控制的线程数/工作量之间的平衡

内存合并与缓存/总线粒度干净地对齐。有 32 个线程时，许多常见访问模式自然就变得"漂亮"

什么是 **warp** 调度器

管理指令发射的单元。

每周期选择就绪的 warp 进行指令发射

处理 warp 级分支和发散 (divergence)

管理一个记分板 (scoreboard)，跟踪哪些 warp 真正就绪

确保源寄存器在发射前已从寄存器堆读出，或已从流水线转发而来

它还要处理结构性限制，比如一个 warp 可以有多少条长延迟操作在途 (in flight)、执行所需的流水线是否可用等

warp 如何执行

warp 调度器扫描分配给 SMSP 的全部 16 个 warp，并挑选 1 个就绪的 warp 取指 -> 把选中的 warp ID + 程序计数器发送给分发单元并发射指令，把指令路由到合适的执行流水线（CUDA 核心、张量核心、加载/存储单元）等待分发反馈warp 调度器总是尽量发射尽可能多的指令，并尽量让所有空闲的 warp 忙起来为了最大化硬件利用率，你通常希望块维度是 4 个 warp 的倍数（例如 128 线程），这样工作才能在四个子分区之间均匀平衡

分发单元

负责每个时钟周期把 warp 的操作实际发送到合适的功能单元这一物理动作，具体做法是向执行流水线发射指令

执行流水线是经由分发端口（dispatch port）通往功能单元的物理数据通路

对于共享的 SM 资源（LSU、SFU），分发单元把指令直接发送到每个执行单元专用的指令队列/流水线。

要继续深入，我们需要理解什么是分发端口

分发端口

分发单元通过它向特定执行流水线（张量核心、CUDA 核心等）发送指令的物理连接点

多个执行流水线竞争同一个分发端口，这意味着它们不能同时接收指令

每个时钟周期只能有一条指令使用共享分发端口；分发单元必须把发射串行化

端口只在指令发射时需要，而不是整个执行期间

L1 指令缓存

它的工作是缓冲可执行的机器码（SASS 指令），让 warp 调度器持续有料可用。

它把执行单元与内存层级解耦。没有它，SM 将不得不从 L2 缓存或 HBM3 内存取指（这需要数百个周期），造成大规模流水线停顿

它针对跳转、循环和函数调用做了优化。当 kernel 循环时（矩阵乘法中很常见），指令直接由 L1 指令缓存（I-Cache）提供，节省带宽和功耗。

线程块如何分布在 **SM** 与 **SMSP** 之间

GigaThread Engine 会选择拥有足够资源来处理你的线程块的那个 SM。

除非块非常小，否则硬件不会把整个块放在单个 SMSP 里。硬件把块拆成 warp，然后以轮询（round robin）方式分配这些 warp。

每当我们使用 `__syncthreads()` 时，同步由所有 SMSP 共享的共享内存中的屏障逻辑处理。其他 warp 会被置于睡眠，直到每个 warp 都完成执行。

PCIe 5.0 主机接口

它是连接 H100 与主机 CPU 和系统内存的主干 128 GB/s 数据高速公路。

它是连接 GPU 与网卡（NIC）的关键物理桥梁。它支持 GPUDirect RDMA，让网卡可以直接读取 GPU 内存而不给 CPU 增加负担。

结论

H100 的决定性特征是转向完全异步的执行。我们从简单的同步拷贝演进到用 TMA 在后台处理数据搬运，让 SM 完全专注于计算。专用化：架构为每个瓶颈都依赖专用单元：TMA 负责内存带宽。第四代张量核心负责矩阵数学。SFU 负责复杂数学数据高效地从 HBM3（高带宽）-> L2 缓存（50MB 统一）-> 共享内存（低延迟）-> 寄存器（最高速度）流动。